FVX Command Templates

FVX Command Templates allow you to program in **FVX** more efficiently. Each of the links below brings you to **FVX** code that can be used to read in data, create surfaces and rakes, and perform other tasks relevant to your visualization that can be immediately copied into your program. Text contained in the example templates can be selected, copied, and pasted into new **FVX** scripts. Where applicable, the sample tables of **FVX** commands are set up to run using the most commonly used or default options with alternate options contained within adjacent commented lines (shown in blue for convenience). To change a field from its present setting, either copy and paste the desired option from the commented line or uncomment the desired line. Field settings shown in red are user-specific or dataset-specific and must be changed for the command to run as intended.

The examples below are NOT a complete listing of all **FVX** commands. For additional information about **FVX**, please see Chapter 4 of the **Reference Manual**.

Reading PLOT3D Data

Reads PLOT3D files into **FieldView**.

```
--Set plot3d data file options
     plot3d_options = {
        format = "binary",  --format: choices include "binary","formatted",
                            -- "unformatted", "dp_unformatted"
        input_mode = "replace",  --input_mode: either "replace" or "append"
        coords = "3d",           --coords: choices include "3d" and "2d"
        multi_grid = "on",       --multi_grid: either "on" or "off"
        iblanks = "on",          --iblanks: either "on" or "off"
        transient = "off",       --transient: either "on" or "off"
        }

--Prepare argument data_input_table
     data_input_table = {
        data_format = "plot3d",
        input_parameters = {

           --xyz file information
           xyz_file = {
           name = "impeller3_xyz",
           grid_point_increment = 1,
           options = plot3d_options,
           },

           --q file information
           q_file = {
           name = "impeller3_abs.q",
           options = plot3d_options,
           },

           --function file information
           function_file = {
              name = "impeller3_fun",
              options = plot3d_options,
              names_filename = "impeller3_nam.nam"
              },
           }, --end input_parameters table
        }  --end data_input_table

--Call function to read dataset
     read_dataset(data_input_table)
```

Reading FieldView Unstructured Data

Reads in **FieldView** Unstructured format.

```
--Prepare unstructured data table
     data_input_table = {
        data_format = "unstructured",
        input_parameters = {
           -- combined grid and results file
           name = "filename",
           options = {
              input_mode = "replace",  --input_mode: "replace" or "append"
              transient = "off"        --transient: either "on" or "off"
              }
           }
        }

--Read unstructured data
     read_dataset(data_input_table)
```

Reading Split FieldView Unstructured and NPARC Data

Reads in split **FieldView** Unstructured and NPARC formats

```
--Prepare split data table
     data_input_table = {
        data_format = "unstructured",
                              --data_format: "nparc/wind" or "unstructured"
        input_parameters = {
             -- grid file information
             grid_file = {
                  name = "gridfile.uns",
                  options = {
                     input_mode = "replace",
                                       --input_mode: "replace" or "append"
                     transient = "off" --transient: either "on" or "off"
                     }
                  },-- end grid file information

             -- results file information
             results_file = {
                  name = "resultsfile.uns",
                  options = {
                     input_mode = "replace",
                                       --input_mode: "replace" or "append"
                     transient = "off" --transient: either "on" or "off"
                     }
                  }-- end results file information
             }-- end input parameters
        }-- end data_input_table

--Read split data table
     read_dataset(data_input_table)
```

Creating a Boundary Surface

Scalar boundary surface

```
     boundary_table1 = {
        dataset = 1,
        -- specify the name of the scalar
        scalar_func = "scalar_name",     --scalar_func: name or "none"
        -- boundary types
        types = {"body","wing"},         --types: name(s), "all", or "none"
        display_type = "smooth_shading"} --display_type: choices also
                                         --include constant_shading,
                                         --faceted_shading and mesh_shading
        }
--Create boundary surface and assign handle to 'surface_handle1'
     surface_handle1 = create_boundary(boundary_table1)
```

Vector boundary surface with thresholding

```
     boundary_table2 = {
        dataset = 1,
        -- specify the name of the vector function
        vector_func = "vector_name",     --vector_func: name or "none"
        threshold_func = "X",            --threshold_func: name or "none"
        threshold_range = {min=2.4,max=2.5},
        types = {"tail"}}                --types: name(s), "all", or "none"
        }
--Create boundary surface and assign handle to 'surface_handle2'
     surface_handle2 = create_boundary(boundary_table2)
```

Complete boundary surface table

```
     boundary_table3 = {
        dataset = 1,
        -- specify name of the scalar function, vector function, or neither
        scalar_func = "scalar_name",     --scalar_func: name or "none"
        vector_func = "vector_name",     --vector_func: name or "none"
        threshold_func = "X",            --threshold_func: name or "none"
        threshold_range = {min=2.4,max=2.5},
        visibility = "on",               --visibility: "on" or "off"
        types = {"tail", "body"},        --types: name(s), "all", or "none"
        display_type = "smooth_shading"  --display_type: choices also
                                         --include constant_shading,
                                         --faceted_shading and mesh_shading
        }
--Create boundary surface and assign handle to 'surface_handle3'
     surface_handle3 = create_boundary(boundary_table3)
```

Creating a Computational Surface

```
--Define computational surface
     comp_table = {
          dataset = 1,
          grid = 2,
          axis = "I",  --axis: "I", "J", or "K"
          I_inc = 1,   --I_inc: Use only if axis = "I"
          I_axis = {   --I_axis: Use only if axis = "I"
             min = 1,
             current = 1,
             max = 20,
             },
        -- J_inc = 1, --J_inc: Use only if axis = "J"
        -- J_axis = { --J_axis: Use only if axis = "J"
        --   min = 1,
        --   current = 1,
        --   max = 20,
        --   },
        -- K_inc = 1, --K_inc: Use only if axis = "K"
        -- K_axis = { --K_axis: Use only if axis = "K"
        --   min = 1,
        --   current = 1,
        --   max = 20,
        --   },

          --Specify name of scalar function
          scalar_func = "scalar_name",     --scalar_func: name or "none"

          --Specify name of vector function
          vector_func = "velocity_name",   --vector_func: name or "none"

          --Specify name of threshold function
          threshold_func = "thresh_name",  --threshold_func: name or "none"
          threshold_range = {
             min = 0.75,
             max = 1.01,
             },
          visibility = "on",             --visibility: "on" or "off"
          display_type = "mesh_shading",--display_type: choices also
                                         --include constant_shading,
                                         --faceted_shading and mesh_shading
          }

--Call function and assign handle to 'cmp'
     cmp = create_comp(comp_table)
```

Creating an Iso-Surface

```
--Define iso-surface
     iso_table = {
        dataset = 1,
        iso_func = {              --iso_func: either table or string (name)
           mode = "point_and_normal",  --mode: can also be "points_in_plane"
           pt1 = {1,0,0},
           pt2 = {1,1,0},
        -- pt3 = {1,1,1}, --pt3: exists if mode = "points_in_plane"
           },
        iso_value = {
           min = -10,
           current = 0,
           max = 20,
           },
        -- specify the name of the scalar function
        scalar_func = "scalar_name",       --scalar_func: name or "none"
        -- specify the name of the vector function
        vector_func = "vector_name",       --vector_func: name or "none"
        -- specify the name of the threshold function
        threshold_func = "thresh_name",    --threshold_func: name or "none"
        threshold_range = {
           min = 2.4,
           max = 2.5,
          },
        visibility = "on",                 --visibility: "on" or "off"
        display_type = "smooth_shading",   --display_type: choices also
                                           --include constant_shading,
                                        --faceted_shading and mesh_shading
        }

--Create iso surface and assign handle to variable 's'
     s = create_iso(iso_table)
```

Creating a Coordinate Surface

```
--Define coordinate surface
     coord_table = {
        dataset = 1,
        -- specify scalar function name
        scalar_func = "scalar_name",     --scalar_func: name or "none"
        -- specify vector function name
        vector_func = "vector_name",     --vector_func: name or "none"
        -- specify threshold function name
        threshold_func = "thresh_name",  --threshold_func: name or "none"
        threshold_range = {
           min = 0.75,
           max = 1,
           },
        visibility = "on",               --visibility: either "on" or "off"
        axis = "X",                      --axis: "X", "Y", "R", "T", or "Z"
        X_axis = {                       --X_axis: Use only if axis = "X"
           min = -6,
           current = -1.1,
           max = 8,
           },
--      Y_axis = {                       --Y_axis: Use only if axis = "Y"
--         min = -6,
--         current = -1.1,
--         max = 8,
--         },
--      R_axis = {                       --R_axis: Use only if axis = "R"
--         min = -6,
--         current = -1.1,
--         max = 8,
--         },
--      T_axis = {                       --T_axis: Use only if axis = "T"
--         min = -6,
--         current = -1.1,
--         max = 8,
--         },
--      Z_axis = {                       --Z_axis: Use only if axis = "Z"
--         min = -6,
--         current = -1.1,
--         max = 8,
--         },
        }

--Create coordinate surface and assign handle to 'crd'
     crd = create_coord(coord_table)
```

Creating Streamlines

```
     seeding_input_table = {
        seed_coord = "XYZ",    --seed_coord: "IJK_real", "XYZ", "RTZ", or
                               -- "IJK_int"
        mode = "add",          --mode: "add" or "seed_a_surf"

        seeds = {              --seeds: Use when mode = "add"
           x = {1.,2.0,3},     --x: Use when seed_coord = "XYZ"
           y = {4.5,5,6.25},   --y: Use when seed_coord = "XYZ"
           z = {7,8,9},        --z: Use when seed_coord = "XYZ"
        -- r = {1,2,3},        --r: Use when seed_coord = "RTZ"
        -- t = {4,5,6},        --t: Use when seed_coord = "RTZ"
        -- z = {7,8,9},        --z: Use when seed_coord = "RTZ"
        -- i = {1,2,3},        --i: Use when seed_coord = "IJK_real/int"
        -- j = {4,5,6},        --j: Use when seed_coord = "IJK_real/int"
        -- k = {7,8,9},        --k: Use when seed_coord = "IJK_real/int"
        -- grid = {1,1,1},     --grid: Use when seed_coord = "IJK_real/int"
           },

--Use the below fields when mode = "seed_a_surf"
     -- seeding_surface = surf_handle,
     -- seeds_to_add = 8,
     -- inc = 1 --Use when seed_coord = "IJK_int" in place of seeds_to_add
        }

--Define streamline display
     streamline_table = {
        dataset = 1,
        -- specify scalar or vector function
        vector_func = "vector_name",  --vector_func: MUST BE SPECIFIED
        scalar_func = "scalar_name",  --scalar_func: name or "none"
        visibility = "on",            --visibility: either "on" or "off"
        seeding = seeding_input_table,
        display_seeds = "on",         --display_seeds: either "on" or "off"
        calculation_parameters = {
           direction = "both", --direction: "forward", "backward", or "both"
           step = 5,
           time_limit = 20,
           release_interval = 10,
           duration = 3
           }
        }
--Create streamlines and assign it to handle rake_handle
     rake_handle = create_streamline(streamline_table)
```

Set Streamline Display

```
     display_attributes = {
        display_type = "spheres",      --display_type: Choices include
                                       --"complete", "filament", "growing",
                                       --"spheres_and_lines", "spheres",
                                       --"lines_of_spheres", "ribbons", and
                                       --"lines_of_dots"

     -- ribbon_width = 32,--ribbon_width: Use when display_type = "ribbons"
        sphere_scale = 1, --sphere_scale: Use when display includes spheres

        scalar_sphere_sizing = "off",    --scalar_sphere_sizing: Use when
                                         --display includes spheres
        animate = "up",                  --animate: "up", "down", or "off"
        animate_divs = 100,
        }

--Set streamline display
     set_streamlines_display (display_attributes)
```

Modifying Display Properties of Surface or Rake

Example: Change the properties of a coordinate surface made through **FVX**
(NOTE: see **page 8** above for complete coordinate surface options)

```
--Define coordinate surface
     coord_table = {
        dataset = 1,
        -- specify scalar function name
        scalar_func = "current_scalar_name",
        -- specify vector function name
        -- vector_func = "none",
        -- specify threshold function name
        threshold_func = "Y",
        threshold_range = {
           min = 0.75,
           max = 1,
           },
        visibility = "on",                --visibility: either "on" or "off"
        axis = "X",                       --axis: "X", "Y", "R", "T", or "Z"
        X_axis = {                        --X_axis: Use only if axis = "X"
           min = -6,
           current = -1.1,
           max = 8,
           }
        }

--Create coordinate surface and assign handle to coord_handle
     coord_handle = create_coord(coord_table)
```

Change the scalar displayed on the surface:

```
--Modify the scalar function name
     modify(coord_handle,{scalar_func = "new_scalar_name"})
```

Integrate All Surfaces

This function is used to integrate a current scalar function across all visible surfaces in the entire data-set.

```
     scalar_function = "scalar_function_name"
     result_a = integrate_all(scalar_function)   --dataset no. not provided
--result_b = integrate_all(scalar_function, 3)   --dataset no. is provided
```

This is the table assigned to the handle `result_a`:

```
dump(result_a)
_____
        integral_type       string     all_surfaces
        sum                 number     157.0387878417969
        scalar_function     string     Density (Q1)
        average             number     0.9994039522105236
        area                number     157.1324462890625
```

Any variable in the table can be assigned to a variable in **FVX**.

Example:

```
sum_of_integration = result_a.sum
dump(sum_of_integration)
```

A dump of `sum_of_integration` will return this result:

```
number 157.0387878417969
```

Integrate a Surface

This function is used to integrate the current scalar function across a previously defined surface. The input is the handle for the surface to be integrated.
(NOTE: see **page 8** above for complete coordinate surface options)

```
--Define coordinate surface
     coord_table = {
        dataset = 1,
        --Specify scalar function name
        scalar_func = "Density (Q1)",
        visibility = "on",                --visibility: either "on" or "off"
        axis = "X",                       --axis: "X", "Y", "R", "T", or "Z"
        X_axis = {                        --X_axis: Use only if axis = "X"
           min = -6,
           current = -1.1,
           max = 8,
           },
        }

--Create coordinate surface and assign handle to 'c'
     c = create_coord(coord_table)

--Integrate scalar function over coordinate surface and assign results the
--handle "int_surface"
     int_surface = integrate_surface(c)
```

This is the table assigned to handle `int_surface`:

```
dump(int_surface)
_____
         integral_type       string     cur_surface
         sum                 number     45.52766418457031
         surface             string     coord_surf
         sum_V_dot_N         number     130.3430633544922
         vector_function     string     Momentum Vectors [PLOT3D]
         scalar_function     string     Density (Q1)
         average             number     0.9736688215272861
         has_surface_normals string     yes
         area                number     46.75888061523438
```

Any value in the table can be assigned to a variable in the **FVX** program using the handle.

Example:

```
int_average = int_surface.average
```

Point Probe with Current Function

This function is used to return the values of functions at a given point for a given dataset. The dataset entry is optional. If dataset is not provided, it defaults to the current dataset.

```
point = {-1.1, 4.5, 1.77}
probe_out = probe_current_functions(point) --without specifying dataset
--probe_current_functions(point, 3)        -- with dataset specified
```

This is the table assigned to handle `probe_out`:

```
dumpall(probe_out)

table: 1C58A5A8
     point                table          table: 1C58A5D0
       1                    number             -1.1
       2                    number             4.5
       3                    number             1.77
     grid                 number         1
     vector               table          table: 1C58A9E0
       value                table              table: 1C520448
         1                    number                 2.973264932632446
         2                    number                 0.004413581918925047
         3                    number                 0.008921979926526547
       func                 string             Momentum Vectors [PLOT3D]
     scalar               table          table: 1C58A770
       value                number             1.00936496257782
       func                 string             Density (Q1)
     iso                  table          table: 1C58A840
       value                number             4.500000953674316
       func                 string             CUTTING PLANE
     IJK                  table          table: 1C58A6A0
       1                    number             17.99485397338867
       2                    number             29.1614875793457
       3                    number             26.17298316955566
     threshold            table          table: 1C58A910
       value                number             1.00936496257782
       func                 string             Density (Q1)
```

Any value in the table can be assigned to a variable in the **FVX** program using the handle.

```
iso_value = probe_out.iso.value
```

Probe with IJK Coordinates

This function is specific to structured grids. It returns a point given three fixed axis-value pairs, or for a line of values, two fixed axis-value pairs and one range. The first input argument is the table `probe_IJK_input`. The second input argument is the grid number. The third argument is the number of the dataset. If dataset is not provided, it defaults to the current dataset.

```
probe_IJK_input = {
   I=10,
   J=10,
   K=14
   }

--call function with dataset no. specified and print results
probe_IJK_result = probe_IJK_current_functions(probe_IJK_input, 1, 1)
dumpall(probe_IJK_result)
```

This is the table assigned to the `probe_IJK_result` handle:

```
table: 1C66B248
     1                    table           table: 1C669888
        vector               table              table: 1C669CC0
           value                table                 table: 1C669D90
              1                    number                  0.6191400289535523
              2                    number                   1.281299948692322
              3                    number                  -1.985399961471558
           func                  string              Momentum Vectors [PLOT3D]
        grid                 number             1
        point                table              table: 1C6698B0
           1                    number                0.04980316758155823
           2                    number                0.3322598338127136
           3                    number                0.1412362158298492
        scalar               table              table: 1C669A50
           value                number                1.636700034141541
           func                 string                Density (Q1)
        iso                  table              table: 1C669B20
           value                number                0.3322598338127136
           func                 string                CUTTING PLANE
        threshold            table              table: 1C669BF0
           value                number                1.636700034141541
           func                 string                Density (Q1)
        IJK                  table              table: 1C669980
           1                    number                10
           2                    number                10
           3                    number                14
     n                    number          1
```

Transient Data Handling

Causes **FieldView** to move to the specified time step while also allowing the user to change some of the parameters of transient sweeping.

```
     transient_table = {      --use either time_step or solution_time
        time_step = 1,
        --solution_time = 1,

     set_transient(transient_table)
```

To sweep through a transient dataset use this:

```
-- Maximum step is shown with 'n' in the table produced with
-- query_transient()
     transient_info = query_transient()
     TimeMax = transient_info.time_step_range.values.n
     for i = 1, TimeMax do
        -- Move dataset forward in time
        print("\nSetting to transient timestep:
"..transient_info.time_step_range.values[i])
        transient_table = {time_step = transient_info.time_step_range.val-
ues[i]}
        set_transient(transient_table)   -- if already at given timestep,
                                         -- FV will not reload that step
     end -- "for" statement
```

Graphing

This function is used to create a 2D plot based on input criteria and an existing dataset. The input argument is the table `graph_input`.

```
     graph_table = {
        title = "Title of Graph",
        grid = {},
        background = "white",  --background: name or hexadecimal value
        plotbackground = "gray",--plotbackground: name or hexadecimal value
        line = {{            --line: can contain more than one line sub-table
           title = "Line 1 values",
           xdata = coord_point,
           ydata = ave_value,
           option = {color = "blue",   --color: name or hexadecimal value
              pixels = "8",        --pixels: "#" (pixels), "#c" (cm), "#m"
                                   --(mm), "#p" (printer points)
              fill = "red",
              symbol = "diamond", --symbol: "square", "circle", "diamond",
                                   --"plus", "cross", "splus", "scross", or
                                   --"triangle"
              mapy = "y",          --mapy: choose "y" for left axis, "y2" for
                                   --right axis
              smooth = "linear"    --smooth: "linear", "step", "natural", or
                                   --"quadratic"
              }},
           },
        x_axis = {
           title = "Radial Distance",
           titlecolor = "black"}, --titlecolor: name or hexadecimal value
           logscale = "off"       --logscale: either "on" or "off"
           },
        y_axis = {             --y_axis: change to/add y2_axis if mapy = y2
        title = " Average Integral of Scalar",
        titlecolor = "black",    --titlecolor: name or hexadecimal value
           logscale = "off"       --logscale: either "on" or "off"
           },
        }

     graph_it = graph(graph_table) --Create 2D plot

--Checks to make sure graph is created
     if graph_it == nil then
        print("Graph not created. Make sure that you do")
        print("not have nil values in your graph table.")
     end
```

Bar Charts

This function is used to create a 2D bar plot based on input criteria and an existing dataset. The input argument is the same `graph_input` table as above with different settings.

```
     bar_graph_table = {
        title = "Title of Bar Graph",
        grid = {},
        background = "white",
        plotbackground = "gray",
        bar = {{
           title = "Bar plot",
           xdata = coord_point,
           ydata = avg_value,
           option = {
              barwidth = 4,
              foreground = "red",    --foreground: name or hexadecimal value
              background = "black",  --background: name or hexadecimal value
              mapy = "y",            --mapy: choose "y" for left axis, "y2"
                                     --for right axis
              }},
           },
        x_axis = {
           title = "Radial Distance",
           titlecolor = "black",    --titlecolor: name or hexadecimal value
           logscale = "off"         --logscale: either "on" or "off"
           },
        y_axis = {
           title = "Average Integral of Scalar",
           titlecolor = "black",    --titlecolor: name or hexadecimal value
           logscale = "off"         --logscale: either "on" or "off"
           },
--      y2_axis = {                 --y2_axis: include if mapy = "y2"
--         title = "Right Axis Title",
--         titlecolor = "black",    --titlecolor: name or hexadecimal value
--         logscale = "off"         --logscale: either "on" or "off"
--         },
     }

     graph_bar = graph(bar_graph_table) --Create 2D bar plot

--Checks to make sure graph is created
     if graph_bar == nil then
        print("Graph not created. Make sure that you do")
        print("not have nil values in your graph table.")
     end
```

Graph Postscript File

The following function allows a graph to be saved as a postscript file. The input arguments to this function are the variable holding the handle for the graph, the name of the output file, and the table specifying the postscript output options.

```
     graph_handle = graph(graph_table)

--Use the following table to set up output specifications
     postscript_output_options_table = {
        center = "on",          --center: either "on" or "off"
        colormode = "gray",     --colormode: "color", "gray", or "mono"
        decorations = "on",     --decorations: set to "off" to force white
                                --background for printing
        landscape = "off",      --landscape: either "on" or "off"
        maxpect = "off",        --maxpect: set to "on" to maximize size of
                                --graph proportionally
        width = "6i",           --width: "#" (pixels), "#c" (cm), "#m" (mm),
                                --"#i" (inches), "#p" (printer points)
        height = "8i",          --height: "#" (pixels), "#c" (cm), "#m" (mm),
                                --"#i" (inches), "#p" (printer points)
        }

--Set the output file name
     outfile = "graph.ps"

--Output postscript file
postscript_output(graph_handle, outfile, postscript_output_options_table)
```

GUI Creation

This function allows users to create their own graphical user interfaces. The input table can be used to add any number of text, button, and slider widgets to the GUI panel.

```
--Define functions to be used by panel operations
     function button_test() print ('Result of button test.') end

--Set panel specifications
     make_panel_input = {
        title = "Demo Panel",
        xloc = 400,
        yloc = 300,
        width = 650,
        height = 200; -->NOTE: end named index, begin numbered index
        { --button widget
           type = "button",
           title = "Button Test",
           action = function() button_test() end
           },
        { --text widget
           type = "text",
           title = "Enter text followed by a carriage return:",
           action = function(self) print ('text is '..self:get()) end
           },
        { --slider widget
           type = "slider",
           title = "Slider Value:",
           value = {
              abs_min = 1,
              current = 50,
              abs_max = 100,
              },
           drag_action = function() print ('current slider value is',
              panel_handle[3]:get().current) end,
           action = function() print ('typed-in slider value is'
              ..panel_handle[3]:get().current) end,
           digits = 4 --digits: specifies number of significant digits
           },
        }

--Create panel
     panel_handle = make_panel(make_panel_input)
```

Set View

The following function provides users with the ability to fully establish **FieldView** view directions and settings in **FVX**. Note that the text of the `set_view` utility must also be copied into any **FVX** script that uses the command.

```
--Obtain dataset_info_table upon dataset read-in
     dataset_1 = read_dataset(data_input_table)

--Create table of view parameters
     set_view_table = {
        dataset_info_table = dataset_1,
        zoom = 1,
        angle_axis = {
           at = {0, 0, 0},
           from = {-1, 1, 1},
           up = {0, 1, 0}
           },
        translation = {
           x = 2,
           y = 0,
           z = 0
           },
        outline = "off",            --outline: "on" or "off"
        perspective = {
           angle = 40,              --angle: Measured in degrees
           z = -2.75
           },
        axis_markers = "on",        --axis_markers: "on" or "off"
        mouse_mode = "track",       --mouse_mode: "track" or "running"
        light_direction = {
           x_light = 0.577350,
           y_light = 0.577350,
           z_light = 0.577350
           },
        rotation_center = {
           x_rot = 0,
           y_rot = 0,
           z_rot = 0
           },
        quick_transparency = "off"  --quick_transparency: "on" or "off"
        }

--Set view
     set_view(set_view_table)
```

Dynamic Clipping

The following function allows users with **FVX** scripts to take advantage of **FieldView**'s Dynamic Clipping feature.

```
--Set up line and visibility parameters
     dynamic_clip_table={
        dataset = 1,
        name = "clipping_line",
        clip_definitions = {
           {point  = {3.27324,  5.43,   4.6723  },
            normal = {3.27324,  18.105, 22.874  }
            },
           {point  = {3.27324,  4.1625, 2.85213 },
            normal = {3.27324,  2.8950, 1.03196 }
            },
           }, --end of definitions
        active = "on",           --active: "on" or "off"
        }

--Activate dynamic clipping
     create_dynamic_clip(dynamic_clip_table)
```